

Book of Specification
SpotiFind



June 2026

Lin Jérôme
EPITA



1 Context and objectives

1.1 Project Context

As part of an ongoing internship at Zelian, this initial project serves as a practical, fast-paced immersion into the company's daily technical stack. Designed as a short-term introductory challenge to kickstart the integration process, the goal is to quickly get up to speed with modern web development standards by building a functional, production-ready mini-application.

To simulate real-world production constraints within a highly compressed timeframe, the project focuses on consuming the external Spotify Web API. This requires implementing robust API token management—securely handling environment variables via a local *.env.local* file without exposing sensitive credentials on version control systems like GitHub.

Instead of a heavy desktop application, this web-based solution relies on Next.js (App Router) and TypeScript to deliver a fast, responsive, and type-safe user experience across both desktop and mobile devices.

1.2 Objectives

The primary objective is to develop Music Finder, a high-performance music exploration platform. Given the intensive and short-term nature of this introductory project, the focus is placed on delivery efficiency, modular architecture, and clean data integration under a tight deadline.

The specific goals are structured as follows:

- **Dynamic Routing & Navigation:**

Implement a seamless, interconnected architecture using Next.js App Router with at least two main pages, including dynamic routing (e.g., */artist/[id]* and */album/[id]*) to explore specific music entities.

- **API Integration & Token Management:**

Establish a secure connection with the Spotify API using the Client Credentials flow. Master the lifecycle of an access token, including secure local storage via *.env.local* and handling mandatory API states (loading skeletons, error handling/token expiration, and empty results).

- **Component-Driven Architecture:**

Build reusable, strictly-typed UI components (such as search bars, generic media cards, and action buttons) using TypeScript props to ensure code maintainability and scalability throughout the rest of the internship.

- **Data Persistence (Bonus Objective):**

To bypass the need for a complex backend or full User OAuth within this short project timeframe, utilize browser *LocalStorage* to allow users to save and manage their favorite tracks locally via a dedicated static page (*/favorites*).

2 Functional description

The system provides an intuitive web interface to search, explore, and organize musical content by consuming the Spotify Web API. It allows real-time data visualization and client-side data persistence.

2.1 Music Exploration & Search

Users can search for tracks, artists, or albums via a central search bar on the homepage. The application fetches data dynamically and displays results categorized into distinct sections. Each result is presented as a typed card component showing essential metadata (e.g., name, artwork, and sub-text).



2.2 Dynamic Fiche Views

The platform handles interconnected navigation using dynamic routing to explore specific entities:

- **Artist Page** (`/artist/[id]`): Displays the artist's profile picture, follower count, their top 5 tracks, and an exhaustive list of their albums.
- **Album Page** (`/album/[id]`): Shows the album cover, release date, artist name, and the complete clickable tracklist.

Clicking on an album card from the Artist page or a result card from the homepage smoothly redirects the user to the corresponding dynamic view.

2.3 Data Persistence & Favorites

To bypass backend storage constraints within this short timeframe, the application utilizes browser `LocalStorage`. A dedicated static page (`/favorites`) fetches and lists all tracks bookmarked by the user. A "heart" icon toggle button on media cards allows instant addition or removal of tracks from local storage.

2.4 User Interface & Experience

The UI is built with a responsive, mobile-friendly component-driven layout. It natively handles three mandatory API states to ensure a seamless UX:

- **Loading State:** Displays skeleton components while fetching API data.
- **Error State:** Shows clear error alerts in case of API token expiration or network failure.
- **Empty State:** Provides explicit feedback when a search query yields no results.

3 Technical constraints

The development process is governed by strict technical limitations to ensure cross-platform compatibility and fast delivery:

- **Framework & Language:** Built using **Next.js (App Router)** and **TypeScript** for robust type safety, component reusability, and native file-system routing.
- **API Consumption:** Relies on the external **Spotify Web API** via the *Client Credentials* flow to manage application-level authentication tokens.
- **Security & Environment:** Sensitive credentials must be stored locally within a `.env.local` file. The actual API key must never be committed to Git; only a template file named `.env.example` is tracked on the repository.
- **Deployment & Optimization:** The application must be structured for easy compilation and hosting, optimized for fluid rendering on both desktop and mobile screens.



4 Project organization

Phase	Micro-Tasks & Deliverables
Setup	Initialization of Next.js project with TypeScript Structure of <code>src/app</code> directory (App Router layout) Creation of <code>.env.local</code> and <code>.env.example</code> templates
Core	Script for Spotify API token acquisition (Client Credentials) Validation of token expiration and lifecycle handler Setting up native global styles and responsive container
Features	Global search bar query handler on homepage (/) Dynamic routing implementation for artists (<code>/artist/[id]</code>) Dynamic routing implementation for albums (<code>/album/[id]</code>)
UX/UI	Creation of reusable strictly-typed media components (<code>Card</code>) Implementation of Skeleton loading screens Implementation of Error banner and Empty State alerts
Bonus	Setup of browser <code>LocalStorage</code> interactions Design of the static bookmarks summary page (<code>/favorites</code>) Activation of the toggle "heart" button on media cards